GECG10069 (561084) F26: Introduction to Programming (C++)

Lab 6 : Repetition Statements



What you will learn from Lab 6

In this lab, you will practice loops and learn how to avoid and debug infinite loops.

TASK 6-1 : FOR VS. WHILE VS. DO-WHILE

- ✓ **for loop**: Best when the number of iterations is known; initialization, condition, and update are all in one line.
- ✓ **while loop**: Condition-driven; the body may not execute at all if the condition is false at the beginning.
- ✓ **do while loop**: Executes the body at least once before checking the condition.
  - ➢ Don't forget the semicolon at the end.

```
#include <iostream>
using namespace std;
int main() {
    const int BEGIN = 1, END = 5;
    // --- for loop ---
    cout << "for loop: ";
    for (int i = BEGIN; i <= END; i++) {
        cout << i << " ";
    }
    cout << endl;

    // --- while loop ---
    cout << "while loop: ";
    int j = BEGIN;
    while (j <= END) {
        cout << j << " ";
        j++;
    }
    cout << endl;

    // --- do while loop ---
    cout << "do while loop: ";
    int k = BEGIN;
    do {
        cout << k << " ";
        k++;
    } while (k <= END);
    cout << endl;

    return 0;
}
```

TASK 6-2 : INFINITE LOOP

✓ If the loop condition never becomes false, the loop runs forever.

```
#include <iostream>
using namespace std;

int main() {
    int i = 1;

    // Missing update → infinite loop
    cout << "Infinite loop example:" << endl;
    while (i <= 5) {
        cout << i << " ";
        // i++;  // ⚠ Forgot to update i
    }
    cout << endl;

    return 0;
}
```

✓ The output result on OneCompiler should look like the screenshot below.

```
Output:

Infinite loop example:
1 1 1 
```

```
for (int i = 1; i != 10; i += 2) {  // never equals 10
    cout << i << " ";
}
```

- Unintended assignment instead of comparison — infinite-loop

```
for (int i = 0; i = 5; i++) {  // i = 5 is assignment, always true
    cout << i << " ";
}
```

✓ Add cout statements inside the loop to trace variable values and check whether they update as expected.

EXERCISE 6-1: MANHATTAN DISTANCE

- ✓ Given an odd positive integer *n*, generate an *n* times *n* grid.
- ✓ For each cell in the grid, calculate its Manhattan distance from the center cell and output the distance value of each cell.
- ✓ The Manhattan distance between two cells (r_1, c_1) and (r_2, c_2) is defined as:

$$D_{Manhattan} = |r_1 - r_2| + |c_1 - c_2|$$

- ✓ where r_i represents the row number of cell i, and c_i represents the column number of cell i.

Input

- ✓ The input consists of one **odd positive integer** *n*, representing the size of the square grid.
  - ➢ $1 \leq n \leq 9$

Output

- ✓ Output an $n \times n$ grid, where each value represents the Manhattan distance from the corresponding cell to the center cell.
- ✓ The values in each row should be separated by a single space, and each row should be printed on a new line.

Sample Input 1
3
Sample Output 1
2 1 2 1 0 1 2 1 2
Sample Explanation 1
For $n = 3$, the center cell is $(2, 2)$. The Manhattan distance of each cell is calculated using: $\|r - 2\| + \|c - 2\|$ Therefore, the center cell has distance 0, its adjacent cells have distance 1, and the corner cells have distance 2.
Sample Input 2
5
Sample Output 2

```
4 3 2 3 4
3 2 1 2 3
2 1 0 1 2
3 2 1 2 3
4 3 2 3 4
```

Sample Code

```
#include <iostream>
#include <cmath>
using namespace std;

int main() {

  // define and read n
  int n;
  cin >> n;

  // calculate the coordinate of center cell
  // 0-based indices are used here: 0, 1, ..., n - 1
  int middle = (n - 1) / 2;

  // output the Manhattan distance of each cell in a nested loop
  for (int i…) {
    for (int j…) {
      int distance = …;
      cout << distance;
      if (j != n - 1)
        cout << " ";
    }
    cout << endl;
  }

  return 0;
}
```

EXERCISE 6-2: MATCH HISTORY ANALYZER

- ✓ Yuuhi is a League of Legends player. After finishing a long gaming session, she wants to analyze her match history and see how well she performed.
- ✓ You are asked to write a C++ program that analyzes the results of her matches, including the total number of wins and losses, the session win rate.

Input

- ✓ The first line contains an integer *N*, representing the number of matches in the gaming session.
  - ➢ $1 \leq N \leq 15$
- ✓ The second line contains *N* characters separated by spaces, each representing the result of one match. **'W'** represents a win, and **'L'** represents a loss.

Output

- ✓ Output the total number of wins, the total number of losses and the win rate.
- ✓ Display the win rate with exactly **2 digits** after the decimal point.
- ✓ Output format:

  Wins: <total number of wins>

  Losses: <total number of losses>

  Win Rate: <win rate>%

Sample Input 1
10 W W L L L W W W W L
Sample Output 1
Wins: 6 Losses: 4 Win Rate: 60.00%
Sample Input 2
8 W L W L W L W L
Sample Output 2
Wins: 4 Losses: 4

```
Win Rate: 50.00%
```

Sample Code

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {

  // define and read N
  int N;
  cin >> N;

  // repeatedly read N match results in a loop statement
  int numWin = 0; int numLoss = 0;
  char currMatch;
  for (…) {
    …
  }

  // calculate the win rate, don’t forget about type casting
  double winrate = …;


  // format output and display


  return 0;
}
```